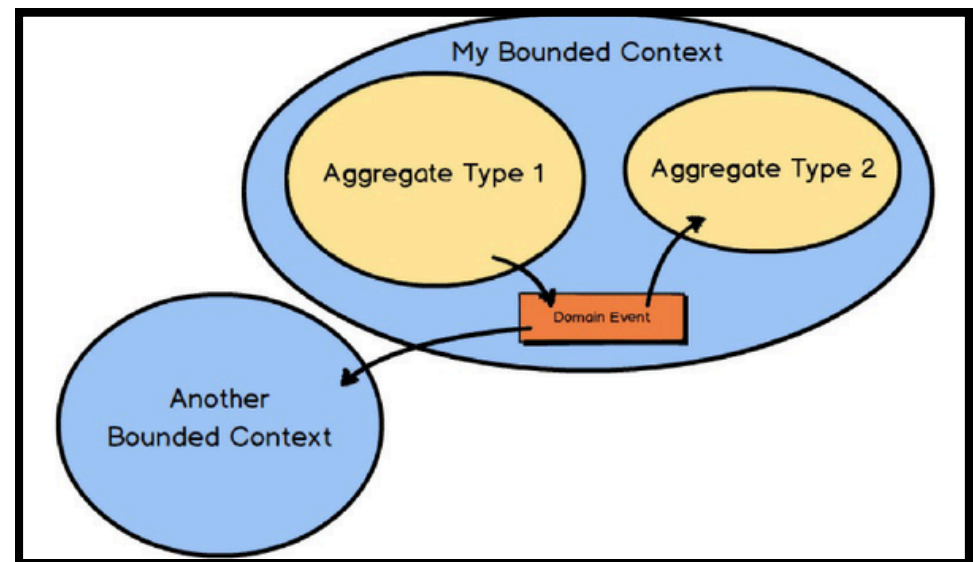


Identificação de Serviços

Domain Driven Design

Domain Driven Design (DDD)

- Representação rica e expressiva do domínio de negócio, focando nas entidades, seus relacionamentos e as regras de negócio
- **Bounded Contexts:** Limites claros dentro de um modelo de domínio.
- **Agregados:** Coleção de objetos (**entidades** e **objetos de valor**)
 - Tratados como uma unidade para fins de dados e transações.



Domain Driven Design (DDD)

- DDD não é apenas sobre organizar código, é sobre comunicação
- **Domain Experts**: Detem **conhecimento das regras de negócio** (e.g., o médico veterinário, o mecânico da loja)
- **Linguagem Ubíqua**: Vocabulário partilhado entre os desenvolvedores e os *Domain Experts*
 - O código reflete a forma do negócio



- `User.updateStatus(true)`
- `Client.changeState(1)`



- `PetOwner.activateAccount()`
- `Vet.scheduleAppointment()`

Entidades e Objetos de Valor

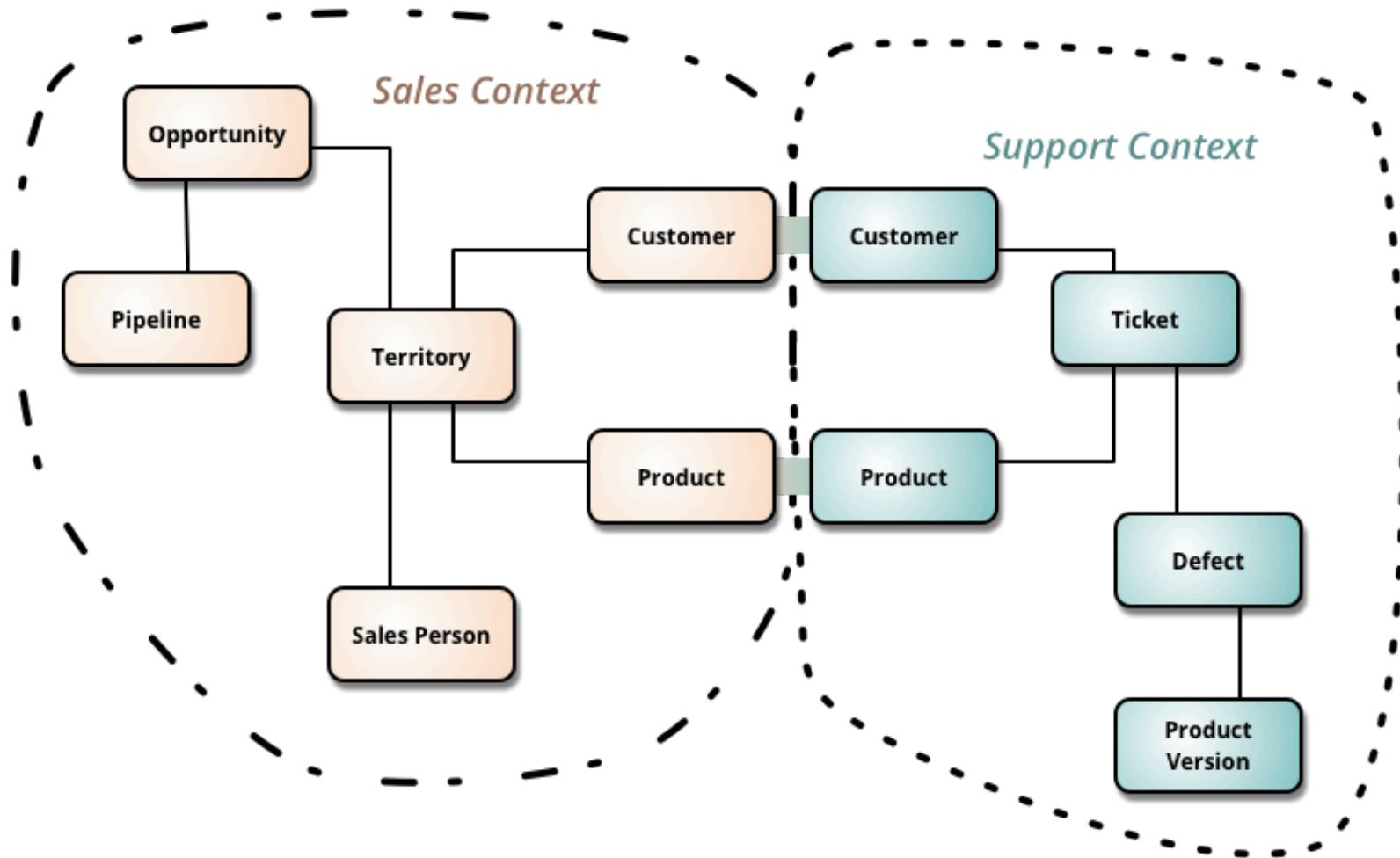
- **Entidade:** Possui uma identidade única durante todo o seu ciclo de vida. *Mutável*
- **Objeto de Valor:** Não tem identidade própria. É definido pelos seus atributos. *Imutável*
 - Substitui-se por um novo em vez de se alterar o atual

```
public record Address(String street, String city, String zipCode) {}

@Entity
public class Vet {
    @Id
    private Long id;
    private String name;
    private Address clinicAddress;

    public void relocate(Address newAddress) {
        this.clinicAddress = newAddress;
    }
}
```

Bounded Contexts e Agregados



Agregados

- Não é apenas um agrupamento de classes relacionadas
 - *Transactional Consistency Boundary*
- **Regra de Ouro:** Modifique apenas um Agregado por transação na base de dados
 - O acesso externo é feito apenas através do *Aggregate Root* (entidade principal do agregado)

```
public class Pet { // Aggregate Root
    private List<Visit> visits;

    public void addVisit(Visit visit){
        if(this.isDeceased){
            throw new DomainException(
                "Error: Pet falecido");
        }
        this.visits.add(visit);
    }

    (...)
}

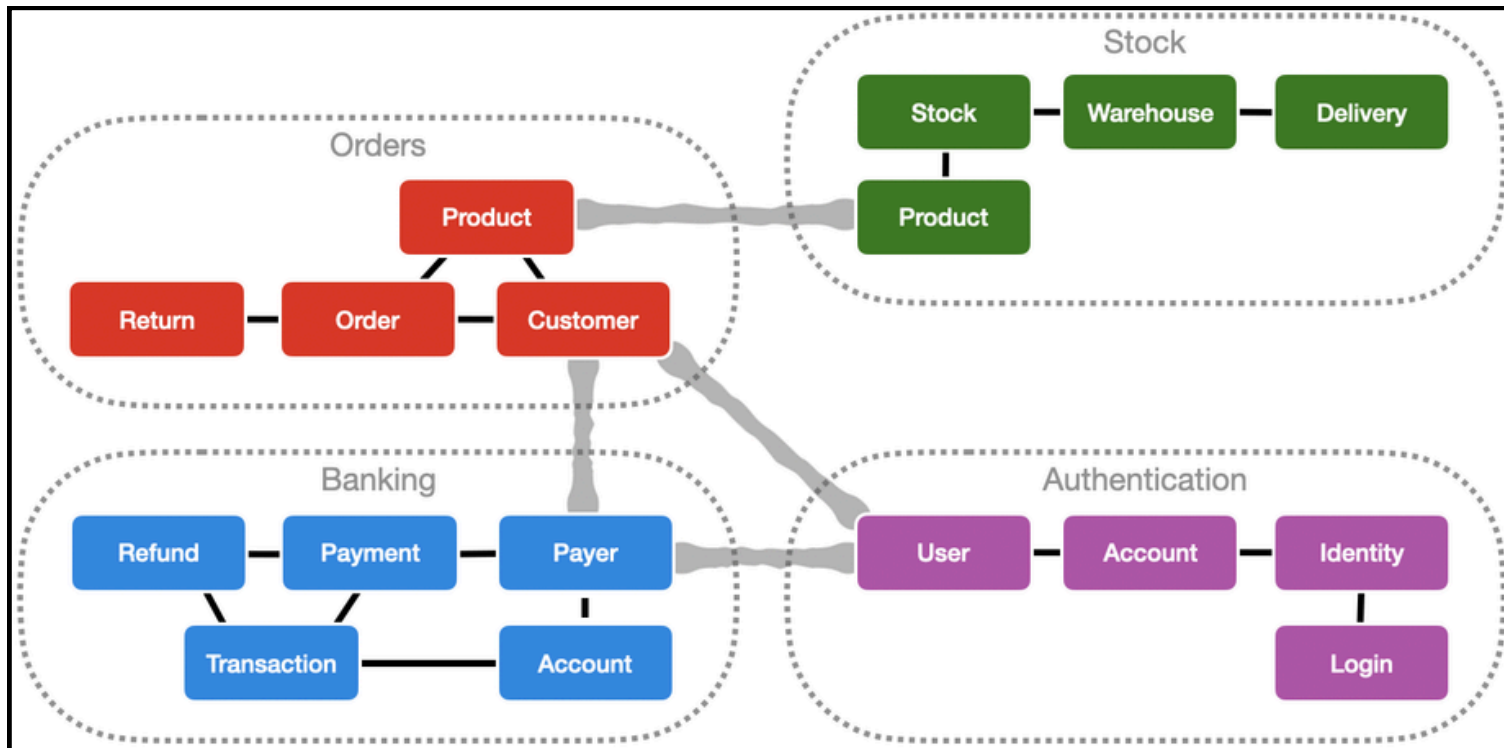
/** Se precisar alterar uma visita: */
visitRepository.save(visit);

//ou

petRepository.save(pet);
```

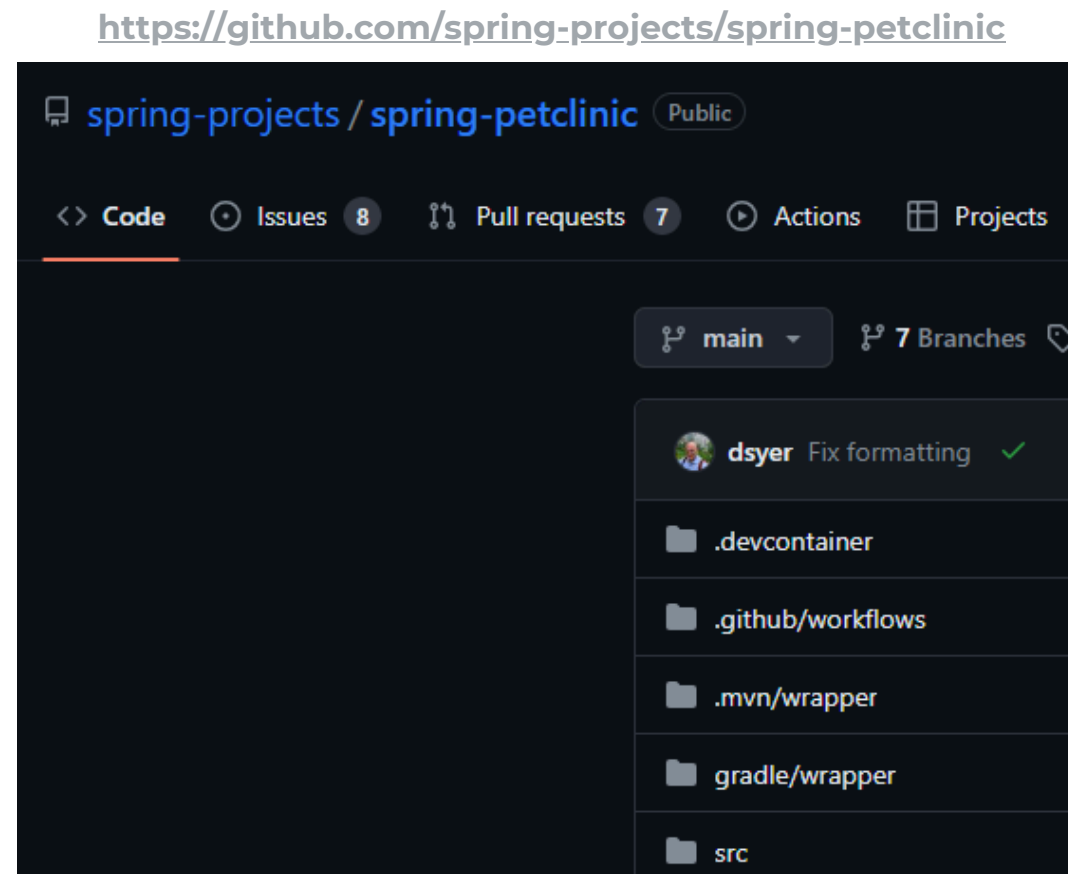
Fronteiras dos Serviços

- **Bounded Contexts** e **agregados** podem ser utilizados como **fronteiras** para os serviços
 - Inicialmente, *Bounded Contexts*
 - ↑abstração → ↓número de serviços
 - Depois migrar para *agregados* (↑número de serviços)



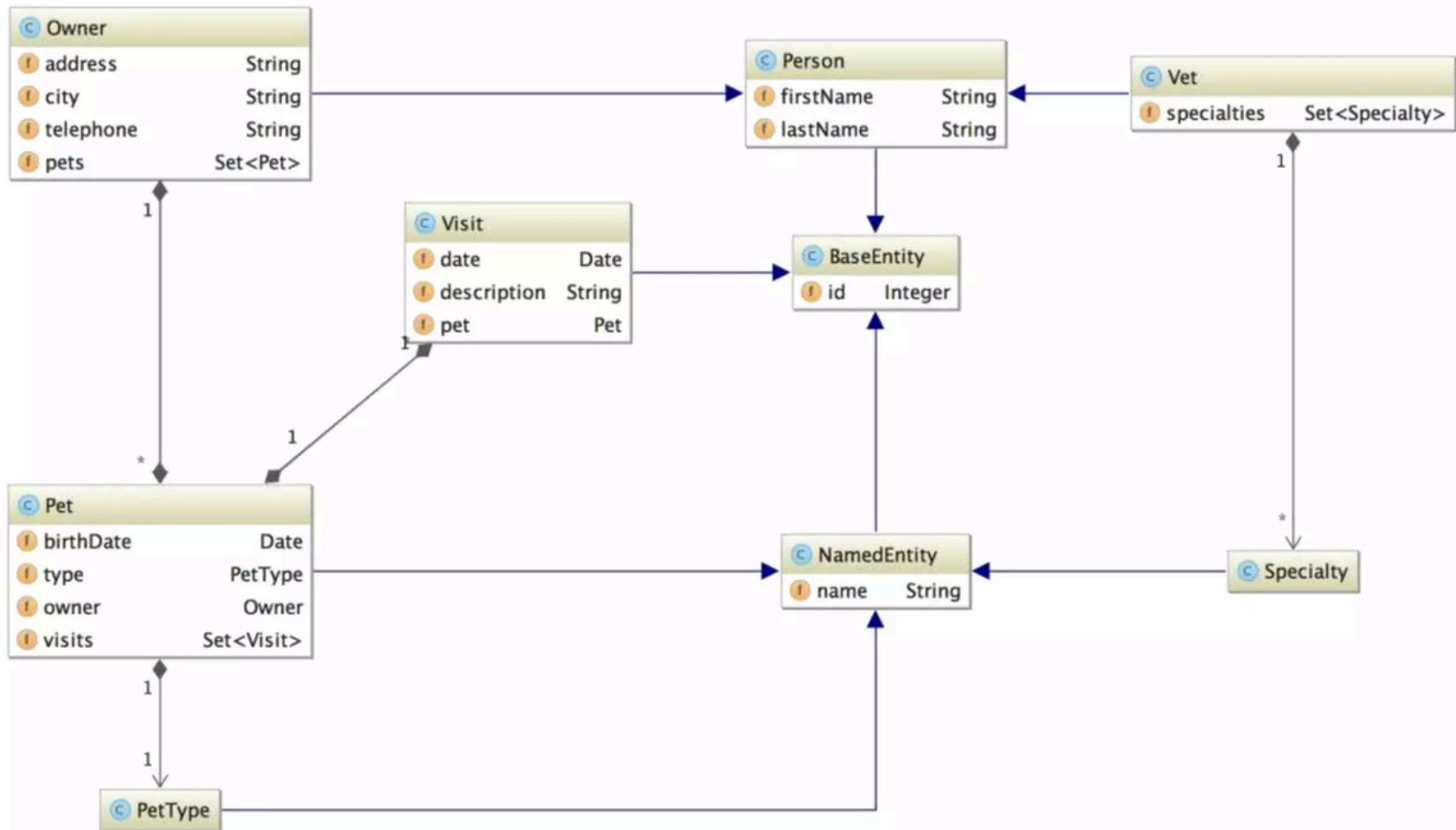
PetClinic

- Aplicação **monolítica** construída usando **Spring**
- Gerenciamento de informações de donos de pets, pets, e visitas ao veterinário.
- Design baseado em uma **arquitetura multicamada**, incluindo camadas de apresentação, serviço, repositório e domínio.



PetClinic

Modelo de Domínio



PetClinic

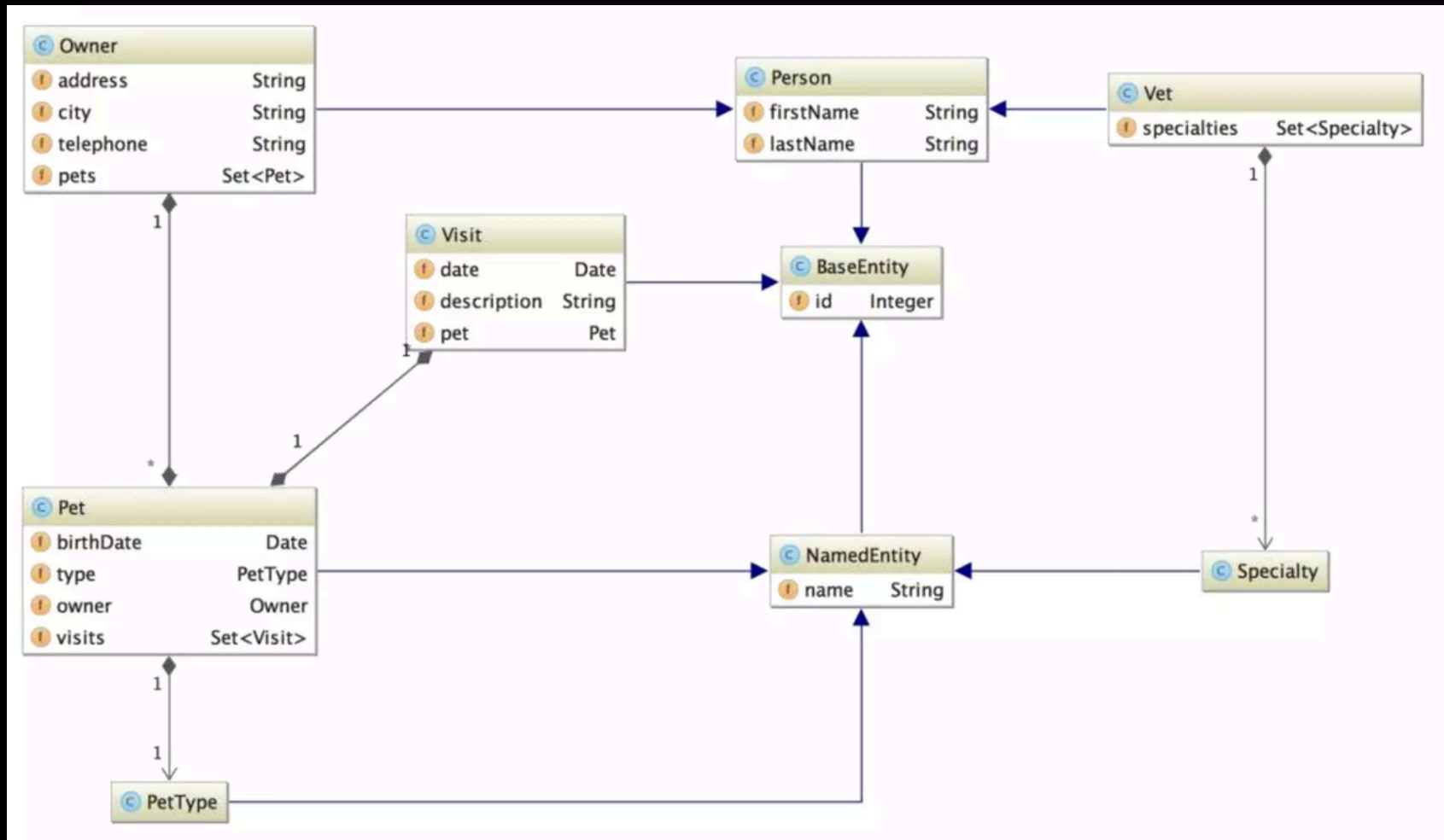
Diagrama de Componentes

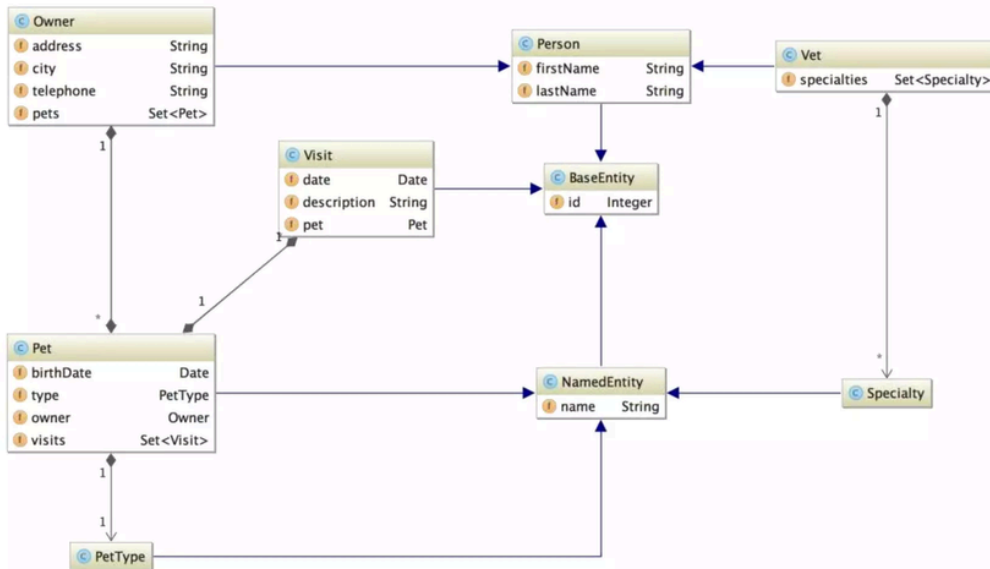
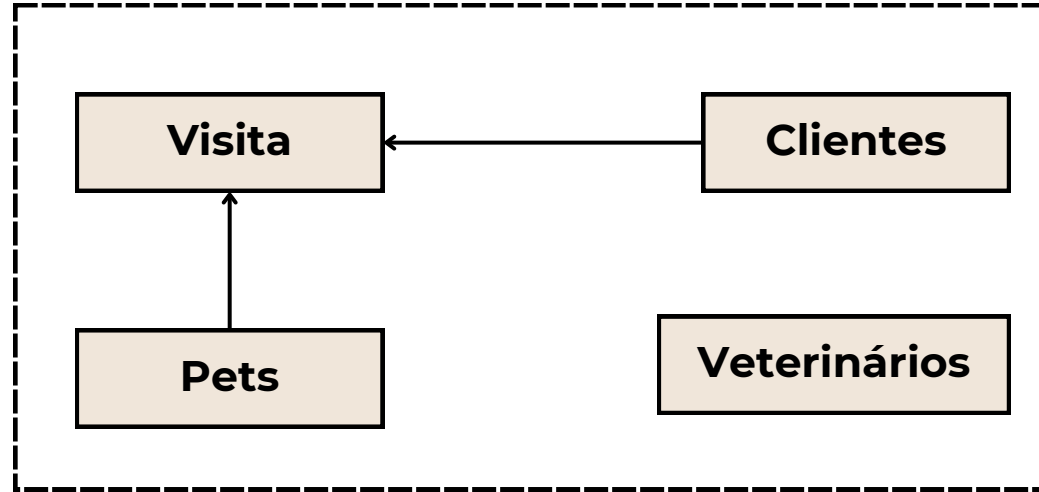


Component diagram for Spring PetClinic - Web Application

The Components diagram for the Spring PetClinic web application.
Sunday 20 November 2016 10:26 GMT

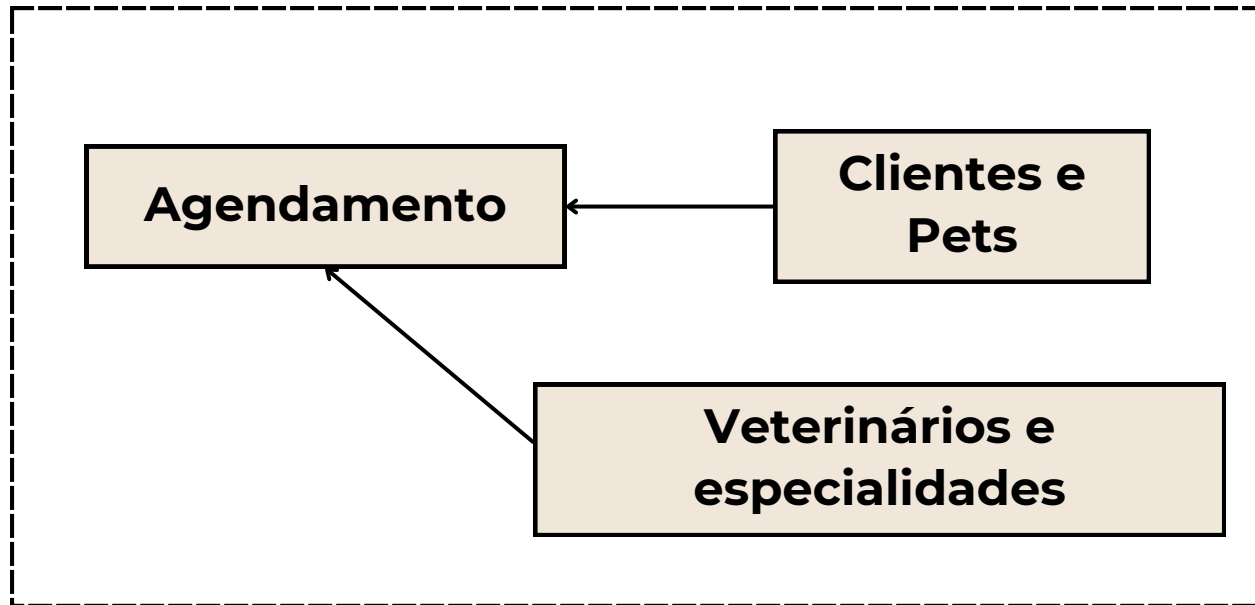
Como vocês dividiriam o PetClinic?





PetClinic

Bounded Context



Planeamento de Migração

- Utilizar DDD é **excelente** para identificar os serviços.
- Alta complexidade na migração:
 - **análise** de código existente, **decomposição de funcionalidades** e **serviços independentes**.
- Migrar por funcionalidades ou pela base de dados.
 - Decompor a **base de dados** é delicado
 - e.g., uma **bd** por microserviço

Duas estratégias possíveis:
Incremental e **Big Bang**

Incremental vs Big Bang

- **Big Bang**

- Coordenação simplificada durante a migração
 - Tudo é feito de uma só vez
- **Alto risco de falha**
- Exigências significativas de tempo e recursos
- Dificuldades com testes abrangentes
- Potencialmente longos períodos de inatividade

Incremental vs Big Bang

- **Incremental**

- **Menor risco de falha**
- Testes mais detalhados
- Menor impacto no negócio durante a migração
 - Ajustes baseados em feedbacks iniciais
- Compatibilidade (sistemas antigos + novos) → esforços adicionais de integração e gestão

Idealmente devemos tentar seguir o modelo incremental

Vamos olhar com mais cuidado para técnicas para realizar a migração nas próximas aulas

Music Corp

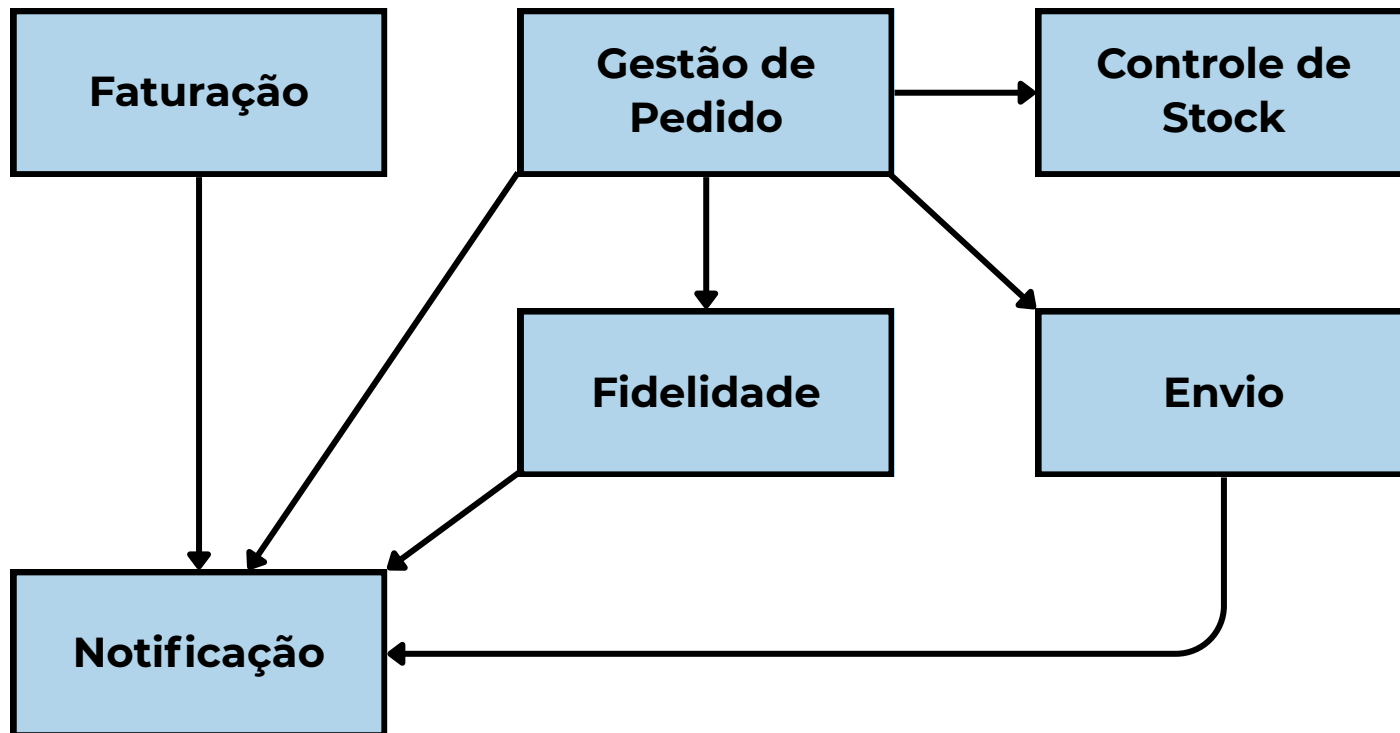
Empresa fictícia usada como exemplo por **Sam Newman** nos seus livros

É uma empresa que gerencia, armazena e vende CDs para os seus clientes



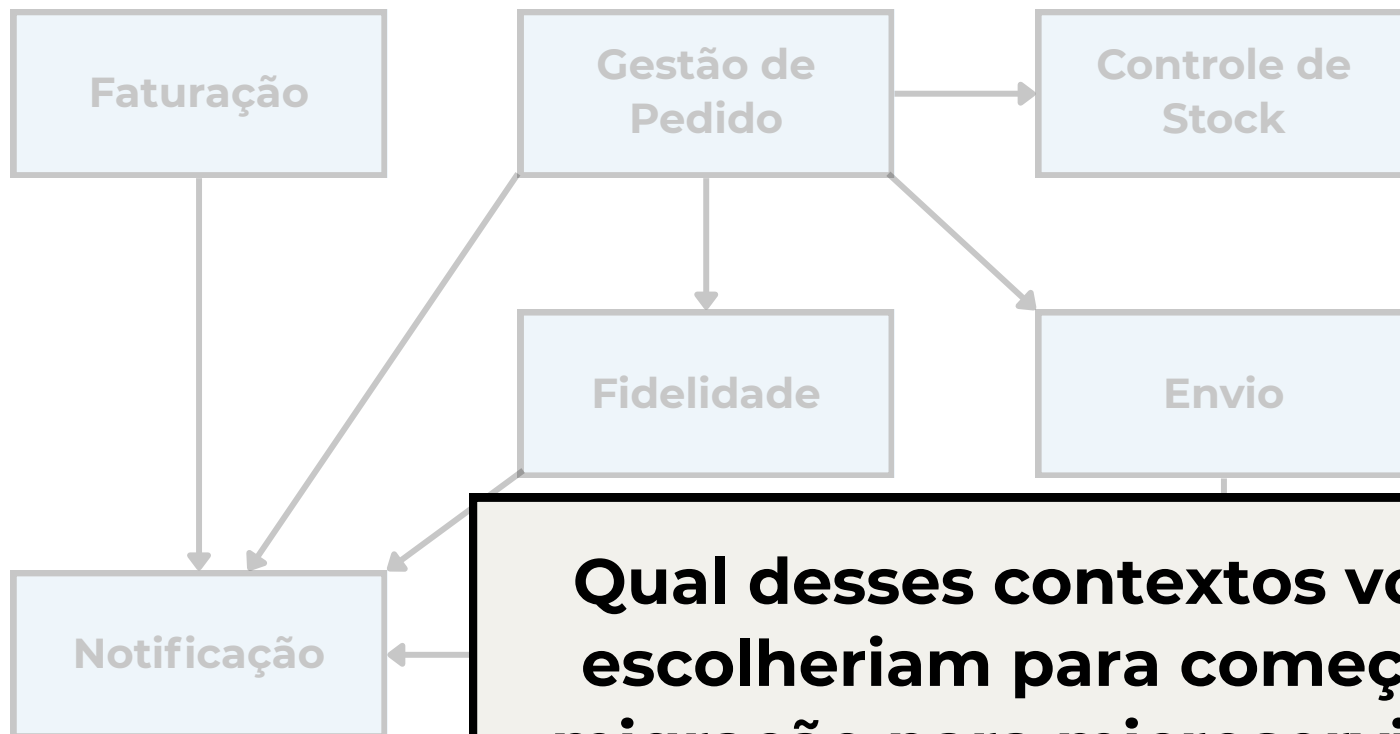
Music Corp

Bounded Context



Music Corp

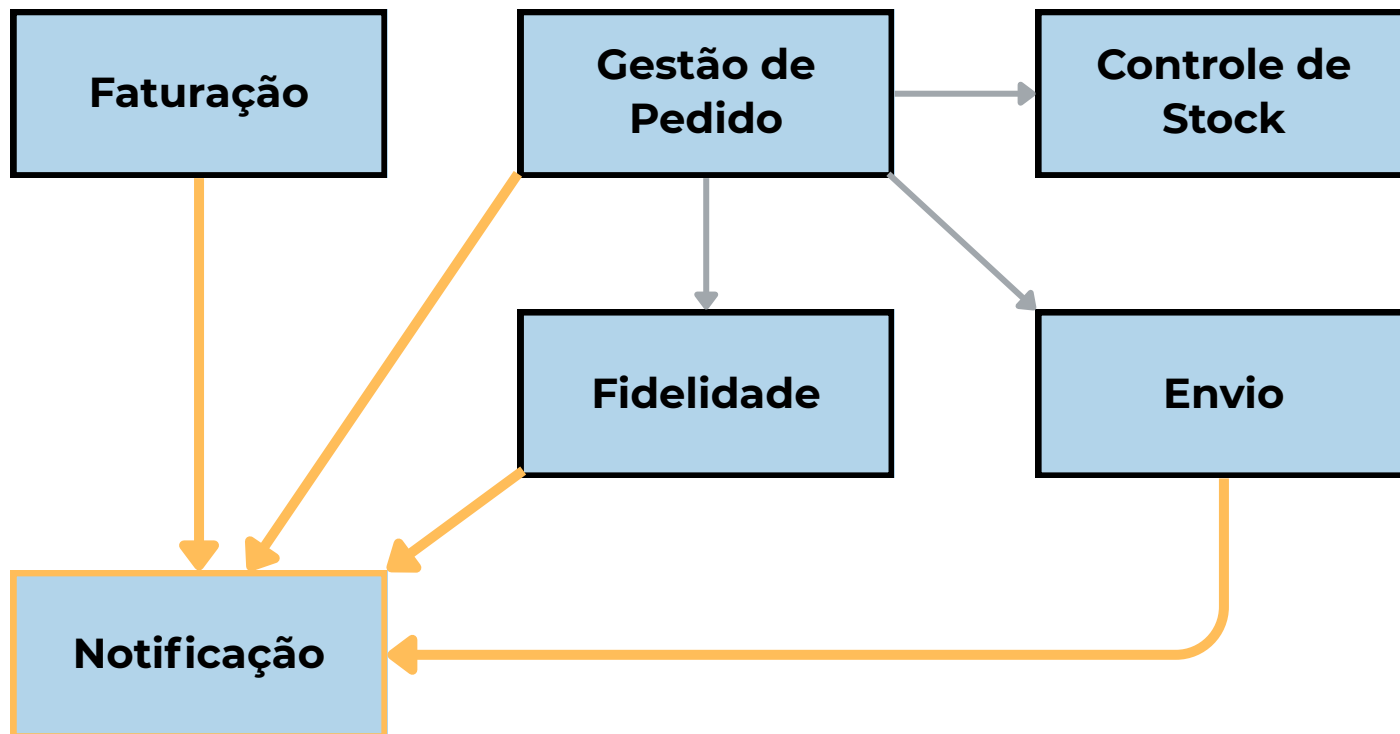
Bounded Context



Qual desses contextos vocês escolheriam para começar a migração para microserviços?

Music Corp

Bounded Context

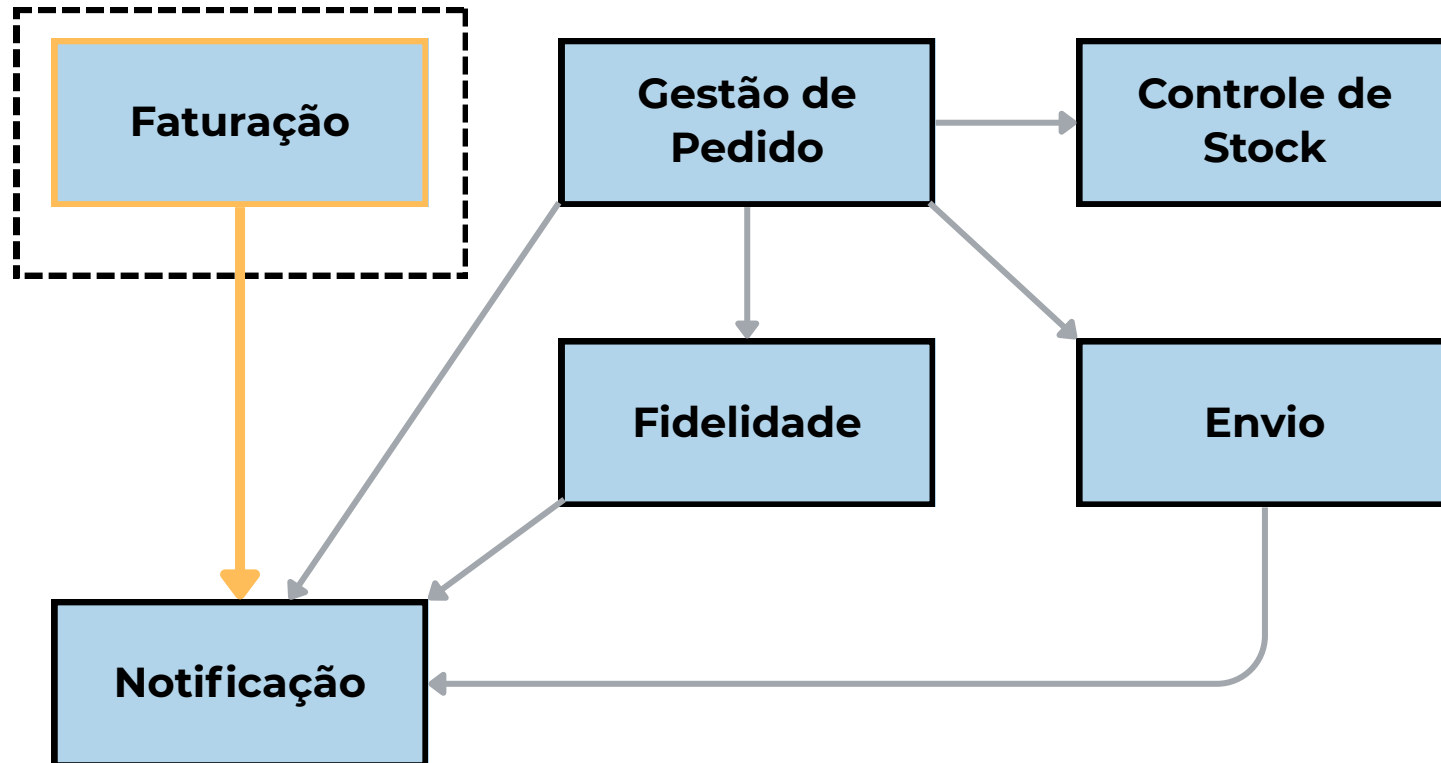


Extrair **Notificação** como um serviço implica* em alterações no código todos esses dependências *upstream*

Music Corp

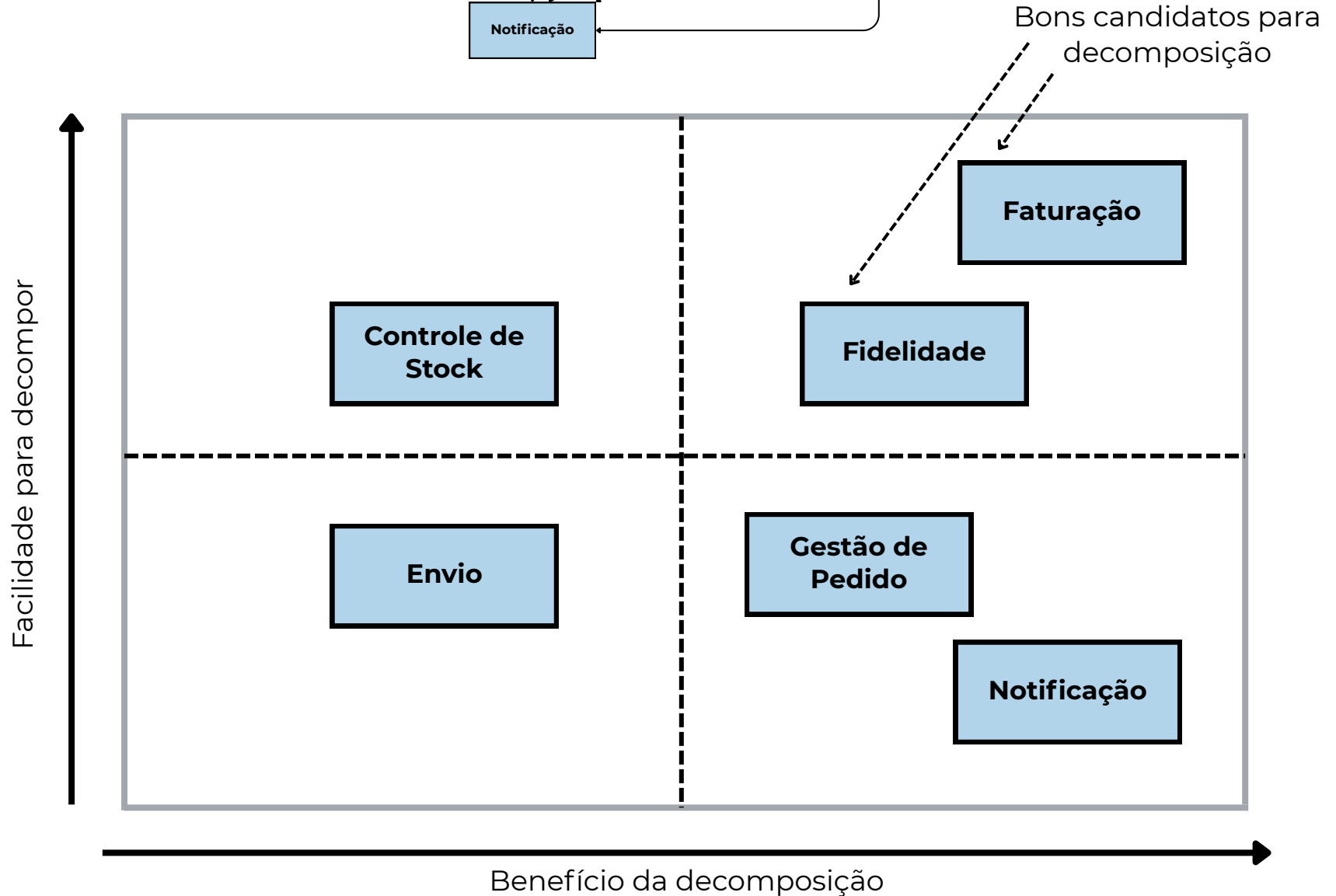
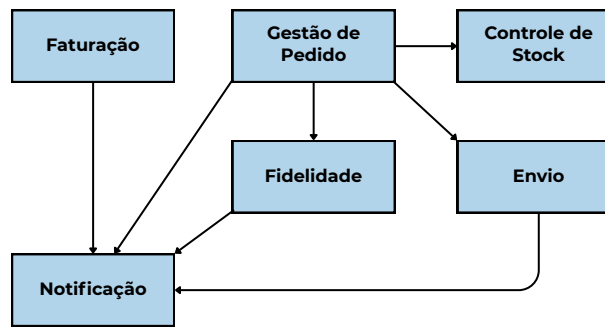
Bounded Context

É possível utilizar padrões para migrar esse serviço com o mínimo de impacto



Music Corp

Alternativa



Desacoplamento

- As dependências upstream (**Faturação**, **Gestão de Pedidos**, ...) de **Notificação** fazem uma invocação direta
 - Bounded context acoplados
- Como extrair o serviço sem ter de alterar o código das dependências upstream?
- Inversão de dependência através de **Eventos de Domínio**

Desacoplamento

Eventos de Domínio

- O serviço de **Faturação** não chama a **Notificação** diretamente
 - Dispara um **evento** referente ao que aconteceu
 - **Notificação** reage a esse evento da forma devida

Desacoplamento

Eventos de Domínio

- O serviço de **Faturação** não chama a **Notificação** diretamente
 - Dispara um **evento** referente ao que aconteceu
 - **Notificação** reage a esse evento da forma devida

```
// Faturamento
public void payInvoice(UUID invoiceId) {
    Invoice invoice = repository.findById(invoiceId);
    invoice.pay();

    // Publica o evento (e.g., Kafka, RabbitMQ, Spring Events)
    eventPublisher.publish(new InvoicePaidEvent(invoice.getCustomerId()));
}

// =====

// Notificação
@EventListener
public void onInvoicePaid(InvoicePaidEvent event) {
    emailService.sendPaymentReceipt(event.getCustomerId());
}
```

Faturamento não
sabe que esse
código existe!

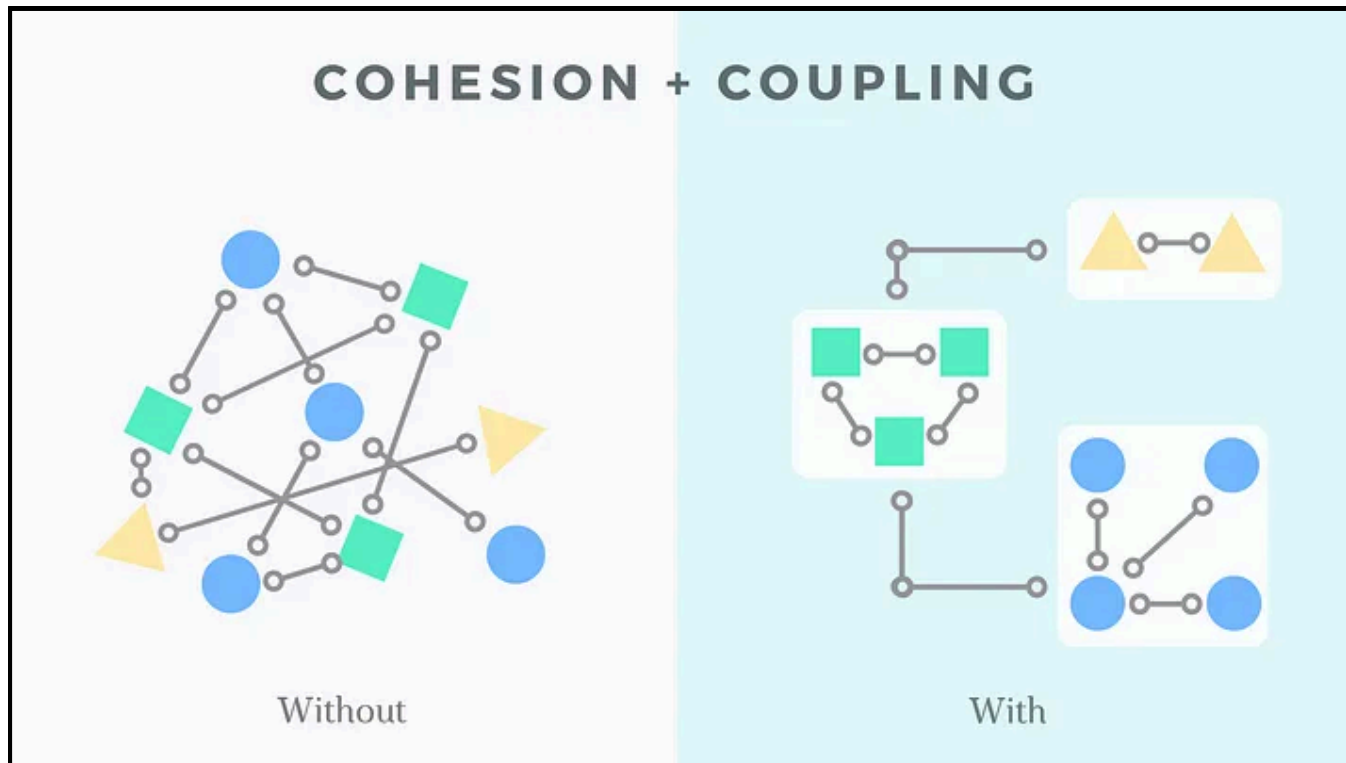
Coesão vs Acoplamento

- **Coesão**

- As coisas que mudam juntas, devem estar todas juntas

- **Acoplamento**

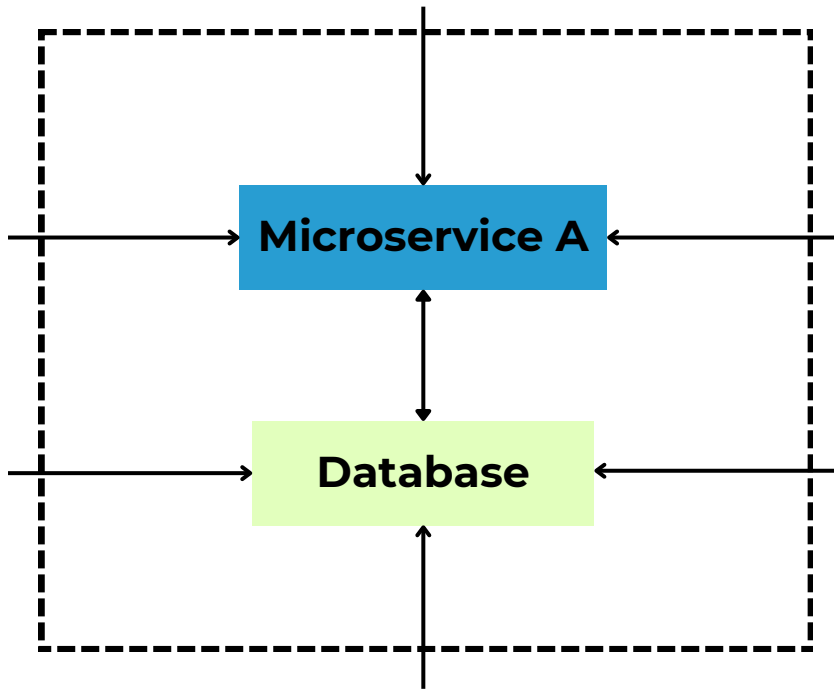
- Se uma modificação for feita em X, isso implica numa modificação em Y



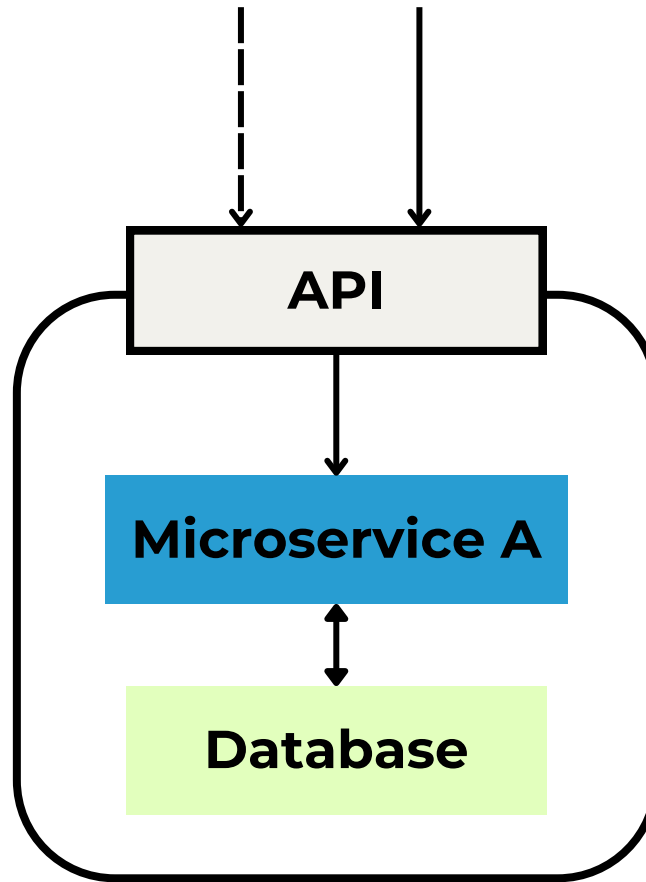
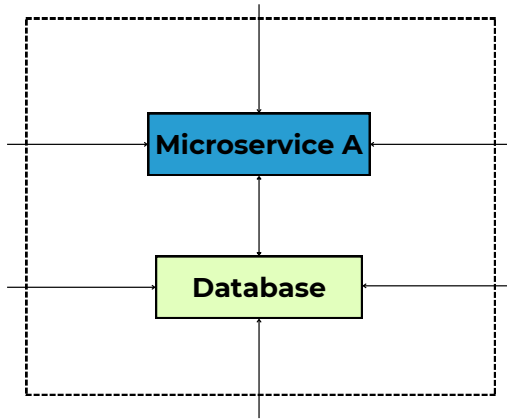
- **Tipos de Acoplamento**

- Conteúdo
- Comum
- Temporal
- Domínio

Information Hiding

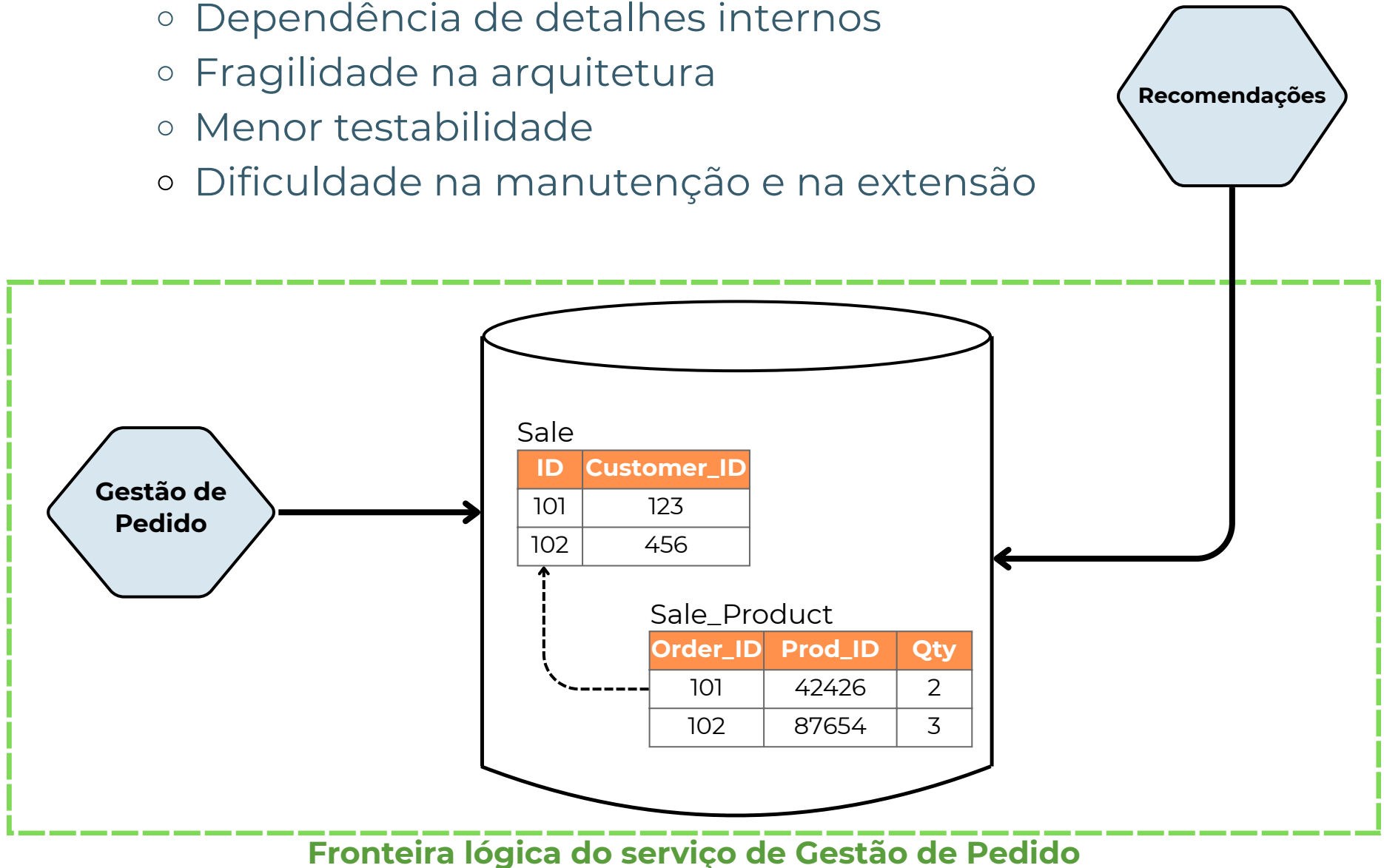


Information Hiding



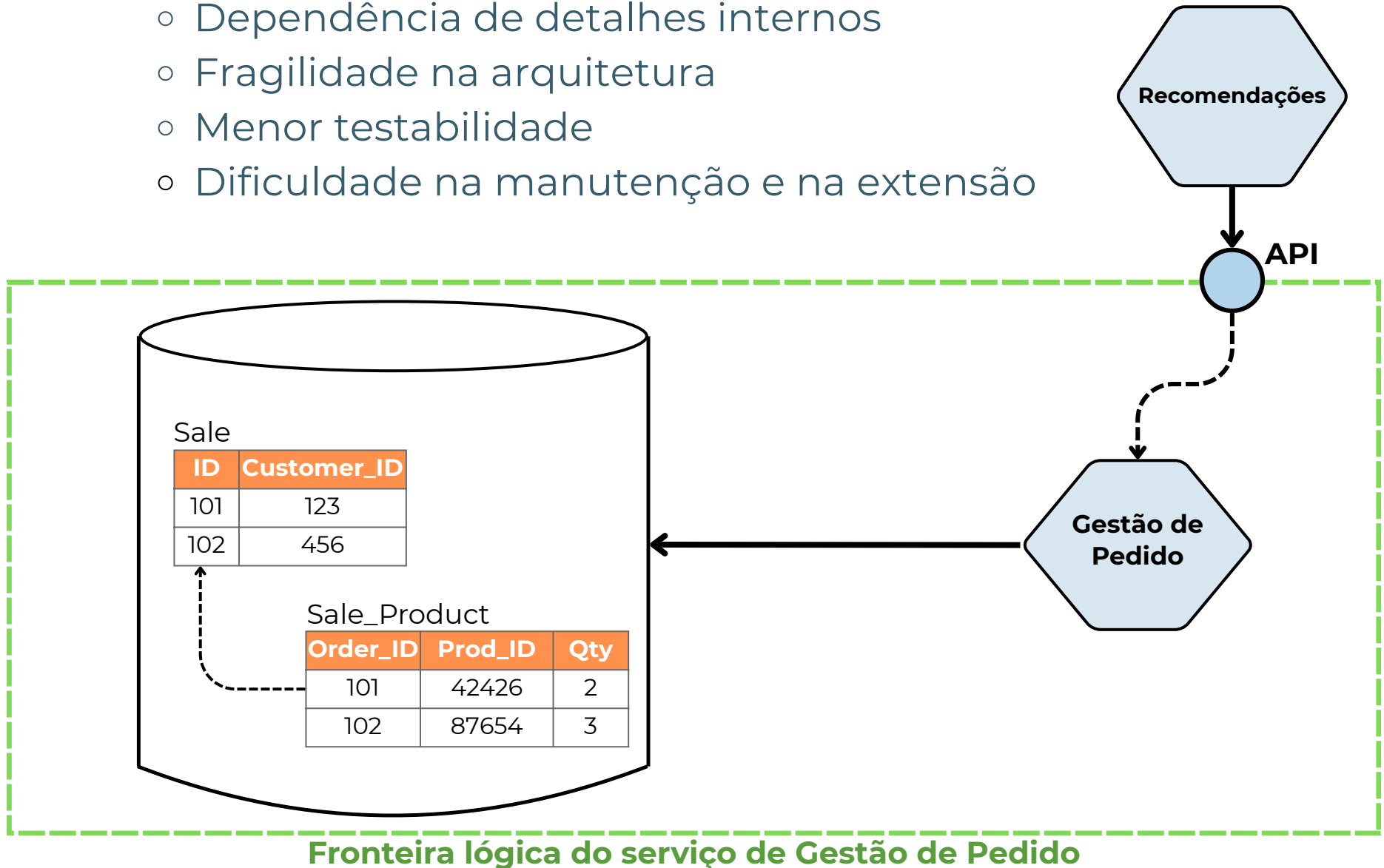
Tipos de acoplamento

- **Acomplamento de Conteúdo**
 - Dependência de detalhes internos
 - Fragilidade na arquitetura
 - Menor testabilidade
 - Dificuldade na manutenção e na extensão



Tipos de acoplamento

- **Acomplamento de Conteúdo**
 - Dependência de detalhes internos
 - Fragilidade na arquitetura
 - Menor testabilidade
 - Dificuldade na manutenção e na extensão



Tipos de acoplamento

- **Acomplamento Temporal**

- Dependência de sequência
- Inicialização e configuração
- Dificuldade na identificação de dependências



Tipos de acoplamento

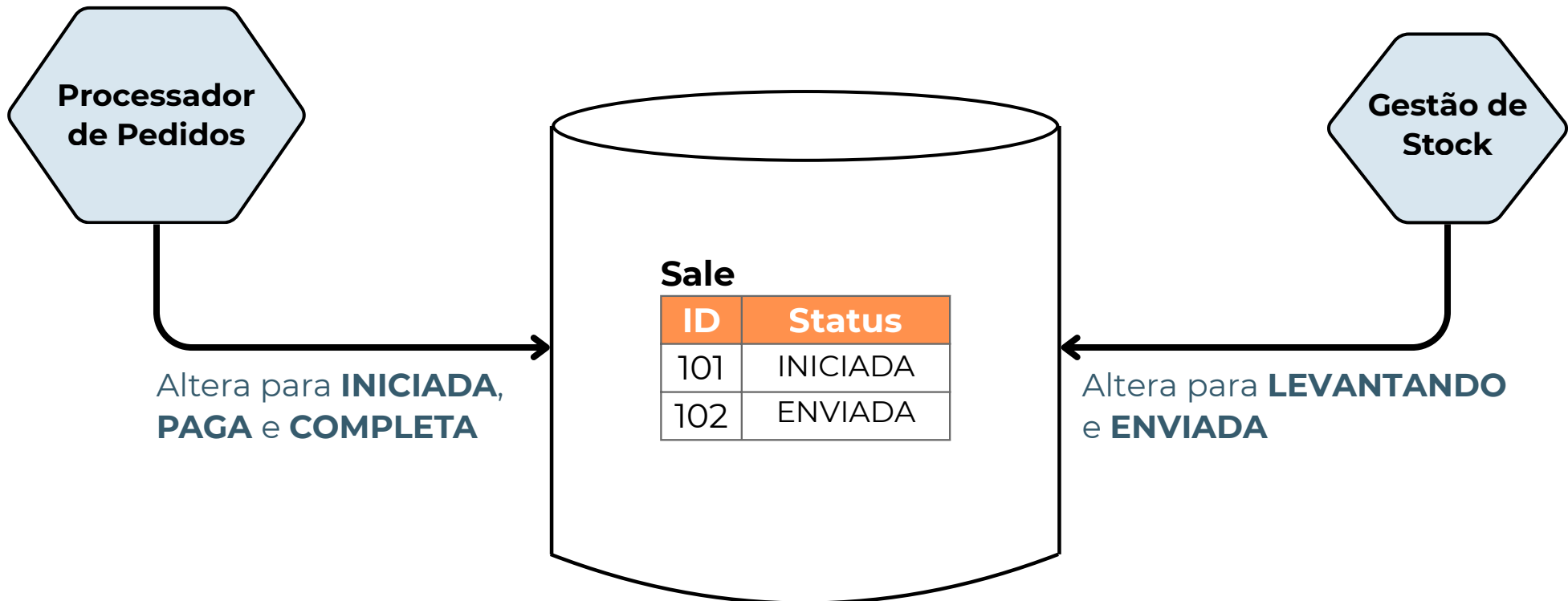
- **Acomplamento Temporal**

- Dependência de sequência
- Inicialização e configuração
- Dificuldade na identificação de dependências



Tipos de acoplamento

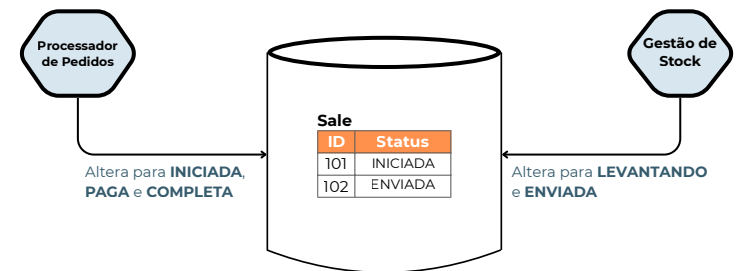
- **Acomplamento Comum**
 - Compartilhamento de recursos globais
 - Alto risco de efeitos colaterais
 - Problemas de concorrência



Tipos de acoplamento

- **Acomplamento Comum**

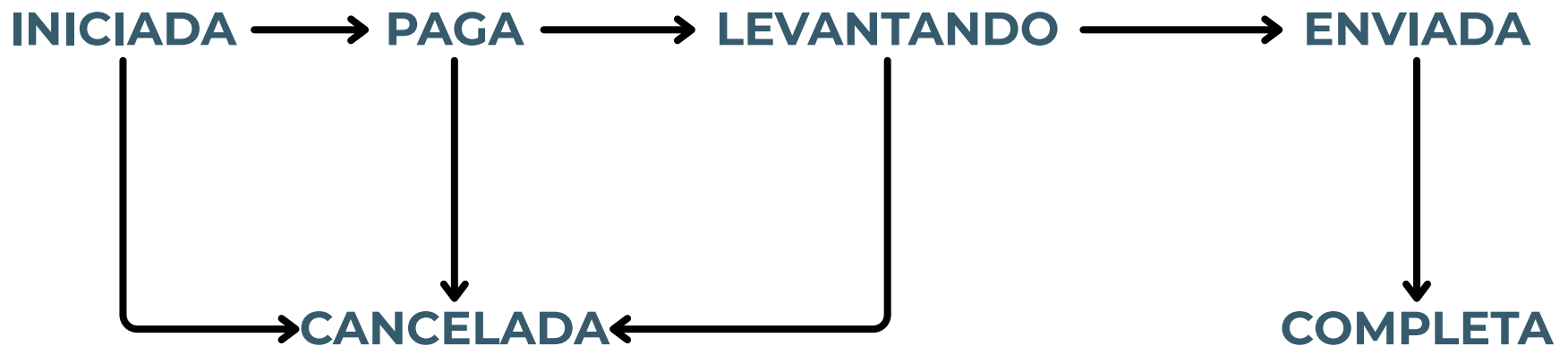
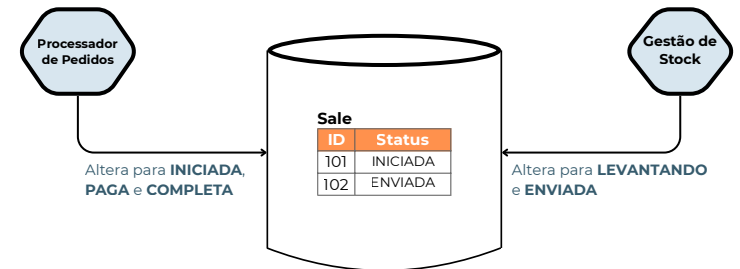
- Compartilhamento de recursos globais
- Alto risco de efeitos colaterais
- Problemas de concorrência



Tipos de acoplamento

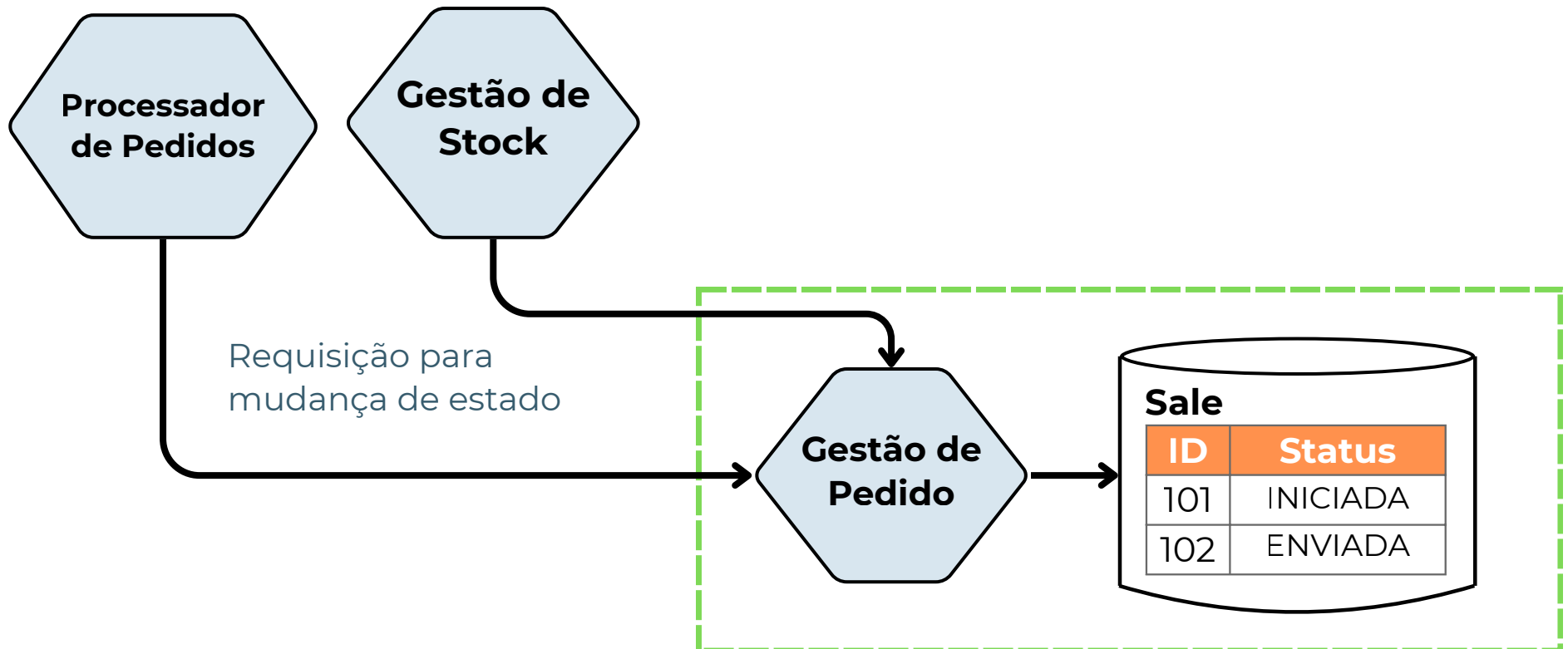
- **Acomplamento Comum**

- Compartilhamento de recursos globais
- Alto risco de efeitos colaterais
- Problemas de concorrência



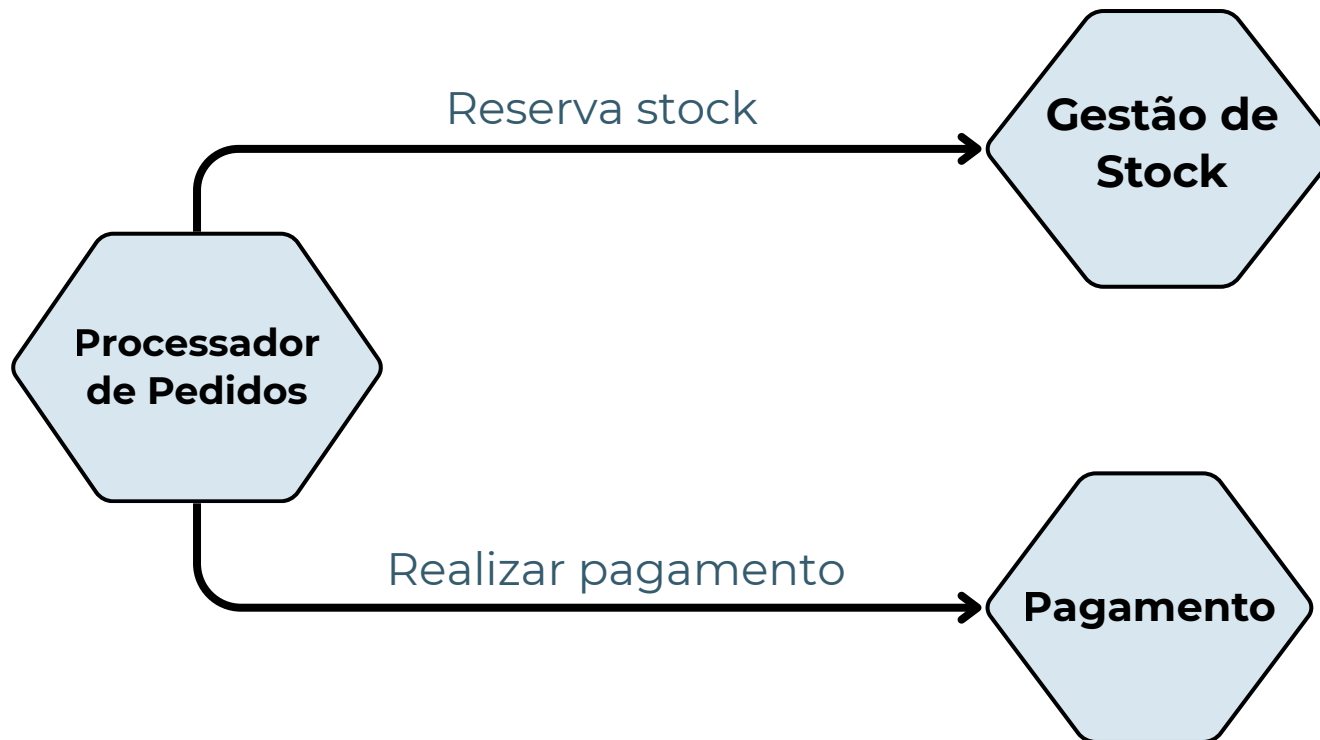
Tipos de acoplamento

- **Acomplamento Comum**
 - Compartilhamento de recursos globais
 - Alto risco de efeitos colaterais
 - Problemas de concorrência



Tipos de acoplamento

- **Acomplamento Domínio**
 - Naturalmente ocorre em projetos com alguma complexidade
 - Coesão de Domínio



Identificação de Serviços

Domain Driven Design